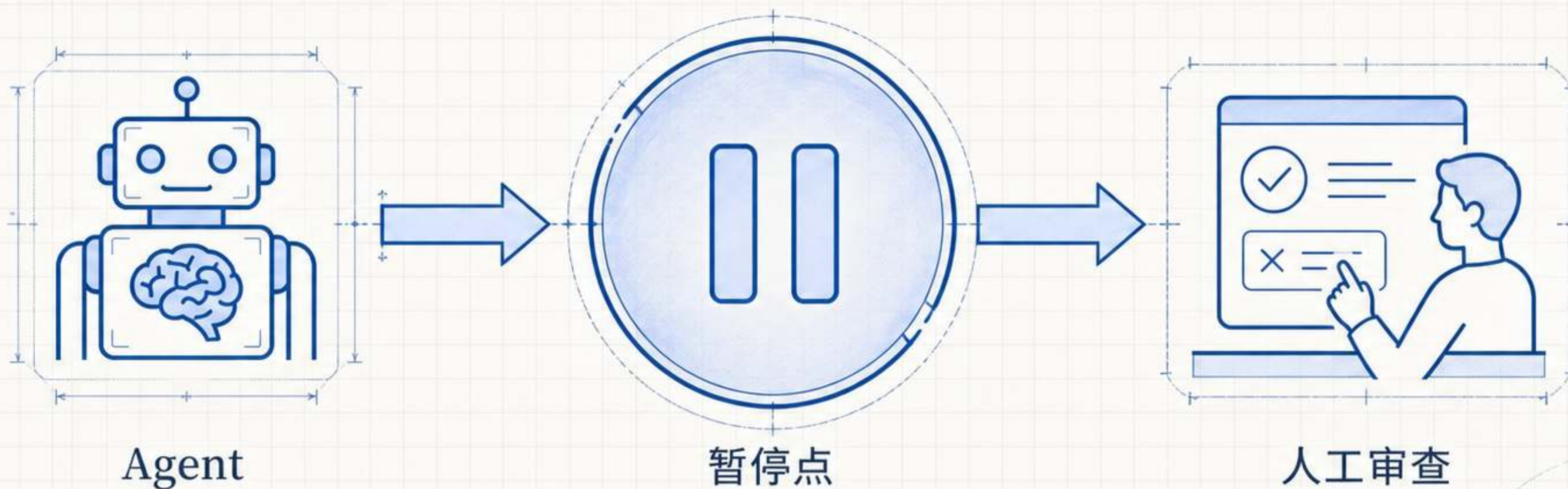


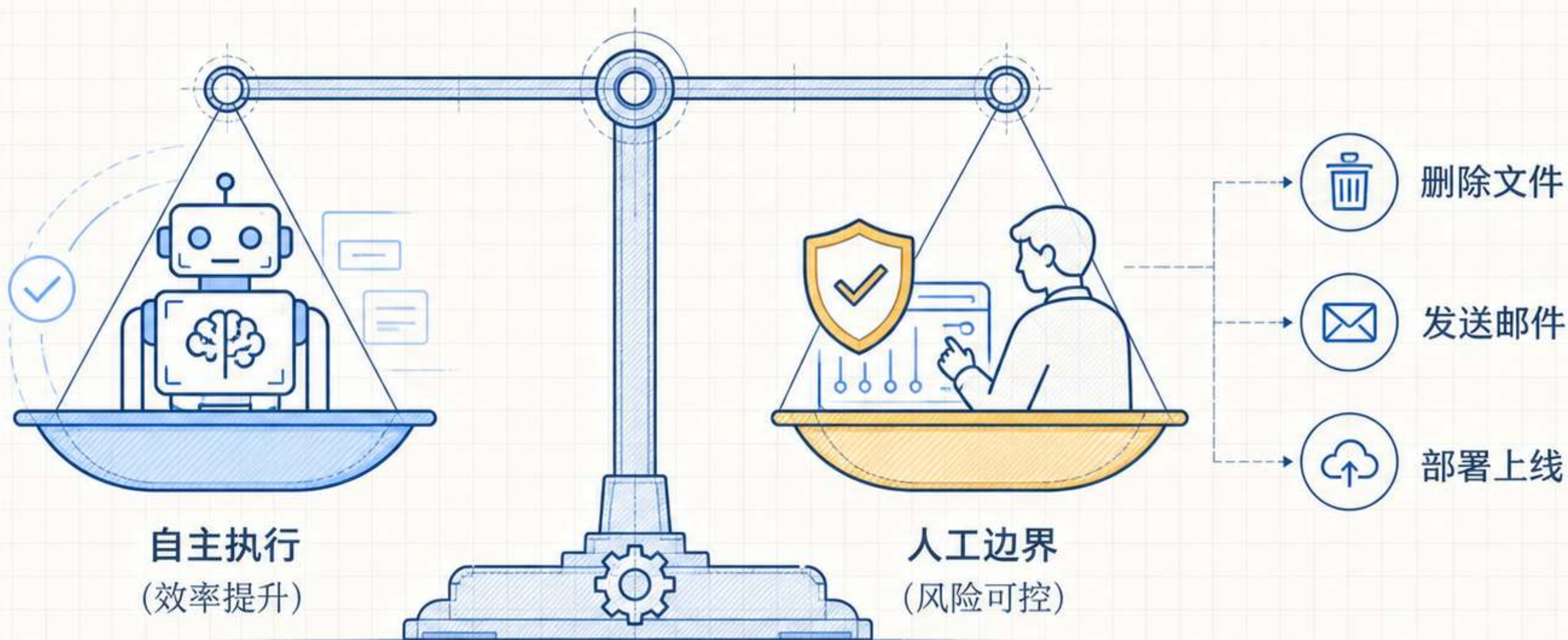
Deep Agents 实战

第 12 讲：HITL 基础 — 配置与决策



为什么需要 HITL?

自主性越高，越需要安全边界



最小可运行配置

先把 checkpointer 和 interrupt_on 接进 Agent

```
checkpointer = MemorySaver()
agent = create_deep_agent(
    model=model,
    tools=[delete_file, read_file, send_email],
    interrupt_on={
        "delete_file": {"allowed_decisions": ["approve", "edit", "reject"]},
        "read_file": False,
        "send_email": {"allowed_decisions": ["approve", "reject"]},
    },
    checkpointer=checkpointer,
)
```



checkpointer :
保存可恢复状态



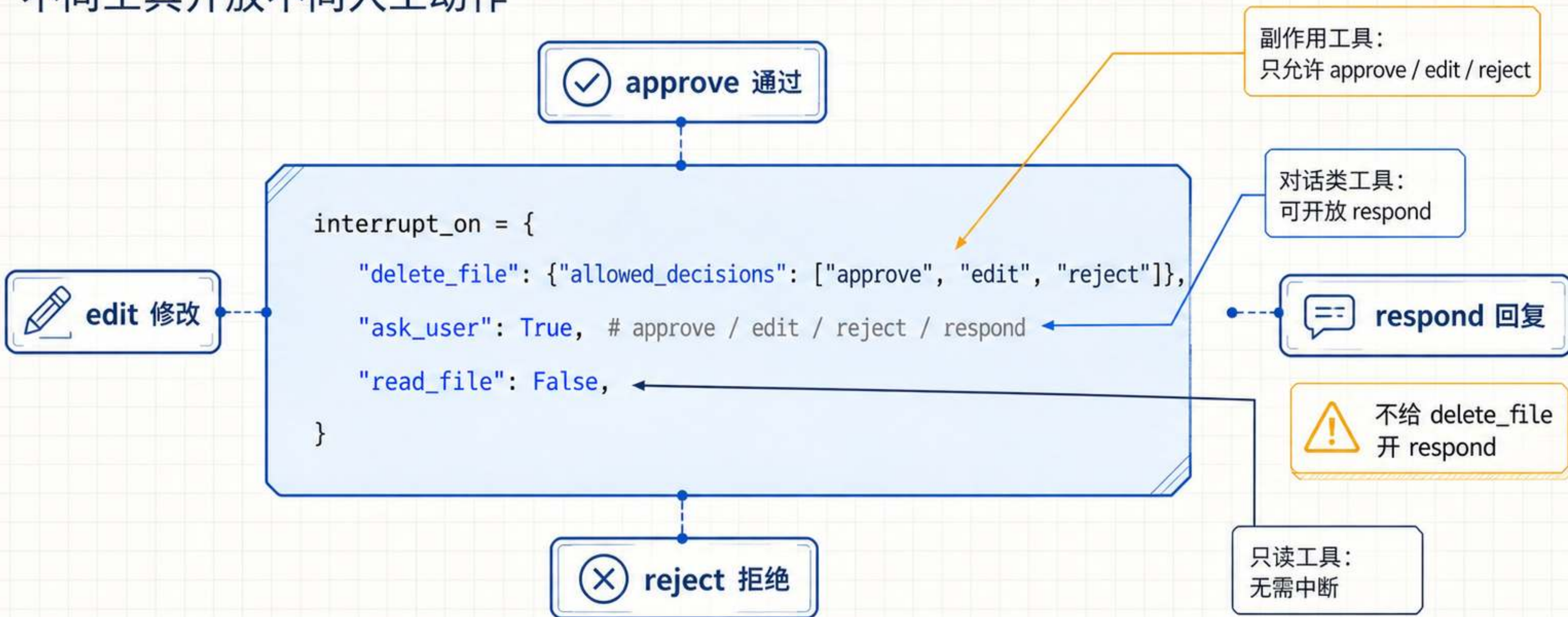
interrupt_on :
按工具配置是否打断



allowed_decisions :
限制人工可选动作

四种决策怎么配

不同工具开放不同人工动作



条件中断：按参数拦截

用 when 只审查真正危险的调用

```
def writes_outside_workspace(request: ToolCallRequest -> bool:
    path = request.tool_call["args"].get("file_path", "")
    return not path.startswith("/workspace/")

interrupt_on={
    "write_file": {
        "allowed_decisions": ["approve", "edit", "reject"],
        "when": writes_outside_workspace,
    }
}
```

ToolCallRequest:
读取工具参数

when:
只拦截命中的调用

/workspace/ 内路行,
外部路径中断

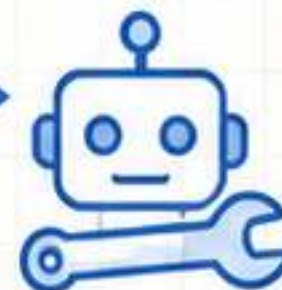
路径门控（按参数拦截）

/workspace/ 下路径
(安全)



放行

执行
工具调用



/workspace/ 外路径
(危险)



中断

人工审查



发起请求：带上 thread_id

恢复时必须回到同一条线程

```
import uuid

config = {"configurable": {"thread_id": str(uuid.uuid4())}}

result = agent.invoke(
    {"messages": [{"role": "user", "content": "删除 temp.txt"}]},
    config=config,
    version="v2",
)
```

生成唯一 thread_id

请求携带同一个 config

首次请求

{ } ≡

thread_id

恢复时复用
thread_id

恢复继续

{ } ≡

读取中断内容

把待审工具展示给用户

</> Python

```
if result.interrupts:
    value = result.interrupts[0].value
    action_requests = value["action_requests"]
    review_configs = value["review_configs"]

    for action in action_requests:
        print(action["name"], action["args"])
        # 展示 allowed_decisions 给用户
```

读取 interrupt value

提取 action_requests

提取 review_configs

展示给用户审批



待用户审批



待审工具

write_file

向文件写入内容



参数 (args)

```
{
  "path": "/reports/summary.md",
  "content": "..."}

```



允许决策 (allowed_decisions)

- approve
- reject



批准



拒绝

恢复执行: Command(resume=...)

decisions 和 action_requests 按顺序对应

</>

```
decisions = [  
    {"type": "approve"},  
    {"type": "reject", "message": "用户拒绝发送邮件, 不要重试。"},  
]  
  
result = agent.invoke(  
    Command(resume={"decisions": decisions}),  
    config=config,  
    version="v2",  
)
```

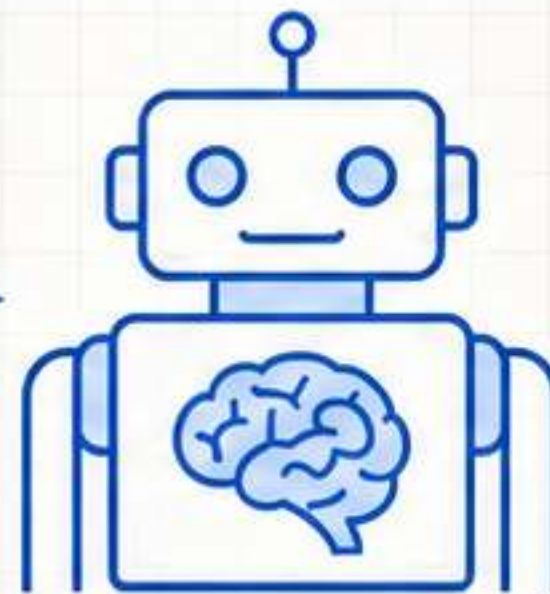
按 action_requests
顺序填写



reject 可带 message



resume 后继续 invoke



Agent

修改参数与子 Agent 配置

实战里常见：改收件人，子 Agent 加严



edit 决策：只改参数，不改工具

```
{
  "type": "edit",
  "edited_action": {
    "name": "send_email",
    "args": {
      "to": "team@company.com",
      "subject": "通知"
    }
  }
}
```



收件人被人工修正



子 Agent 可单独加严

```
subagents=[{
  "name": "file-manager",
  "interrupt_on": {
    "read_file": True,
    "delete_file": True
  }
}]
```



read/delete 都需要确认

文件系统权限中断

保护敏感路径，也可以放到 agent 配置里

```
from langgraph.checkpoint.memory import MemorySaver

agent = create_deep_agent(
    model=model,
    permissions=[
        FilesystemPermission(
            operations=["write"],
            paths=["/secrets/**"],
            mode="interrupt",
        )
    ],
    checkpointer=MemorySaver(),
)
```



1. operations: 只拦截写入

仅拦截危险的写入操作，读操作不受影响



2. paths: 保护 /secrets/**

仅对敏感路径生效，其他路径正常访问



3. mode="interrupt": 暂停并等待确认

触发时中断执行，等待人工确认后再恢复



配置门禁



展示中断



Command(resume=...) 恢复